

Grundlagen digitaler Systeme WS 25/26

Hausaufgabe 3

Besprechung: 13.01.2026

Bitte geben Sie in den folgenden Aufgaben ihre Lösungsschritte stets explizit an, falls nicht anders angegeben. Nicht kommentierte Lösungen werden **nicht** gewertet.

Aufgabe 1 *Assembly und Pseudoinstruktionen*

Gegeben ist folgendes Assembler-Programm:

```
1      .data
2      zahl: word 0x09fa00ce
3
4      .text
5      beginning:
6          lh t0 zahl
7          srli t0 t0 4
8          li t1 0
9      middle:
10         add t2 t2 t0
11         addi t0 t0 -1
12         bne t0 t1 middle
13     end:
14         nop
```

Bei den folgenden Aufgaben kann es Ihnen helfen, Ripes zu nutzen (vorgestellt in Kapitel 9). Nutzen Sie dafür den Single-Cycle-Prozessor RV32IM.

1. Grenzen Sie *Instruktionen* und *Pseudoinstruktionen* voneinander ab. Welche Funktion haben *Segmente* in einem Assembler-Programm?
2. Identifizieren Sie alle Pseudoinstruktionen des Programmes und geben Sie an, in welche Instruktionen sie aufgeschlüsselt werden. Erklären Sie, warum die Pseudoinstruktionen so übersetzt werden.
3. Was machen die einzelnen Segmente dieses Programmes? Welche Bedeutung haben die jeweiligen Register dabei? Welche Funktion wird durch das Programm berechnet?

4. Das Programm erhält einen Wert als Eingabe und gibt einen Wert als Ausgabe zurück. Stehen Ein- und Ausgabe in Registern oder im Speicher? An welcher Stelle?
5. RISC-V benutzt little-endian. Würde sich das Ergebnis ändern, wenn eine andere Endianess benutzt werden würde?
6. Schreiben Sie das Programm so um, dass weniger Instruktionen und temporäre Register benötigt werden.

Aufgabe 2 RISC-V

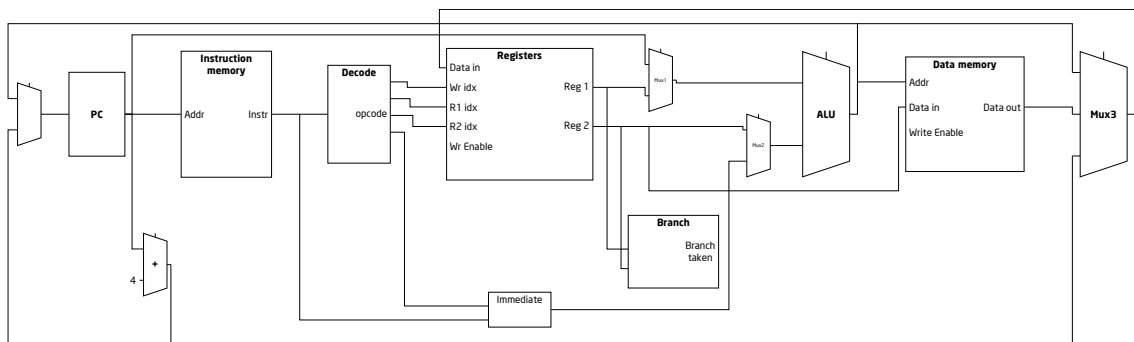


Abbildung 1: RISC-V-Architektur (Bild aus den Vorlesungsfolien)

1. Betrachten Sie folgende Instruktion aus dem in Aufgabe 1 gegebenen Assembly-Code:

Zeile 6: `lh t0 zahl1`

Im Testat werden wir uns diese Instruktion und ihre Ausführung genauer ansehen. Bitte bereiten Sie sich entsprechend vor. Sie sollten z. B. erklären können, warum welcher Wert an den verschiedenen Leitungen in der RISC-V-Architektur anliegt.

2. Im Gegensatz zu anderen Immediate-Instruktionen, die i.d.R. 12 Bit entgegennehmen können, kann `auipc` 20 Bit entgegennehmen. Warum braucht `auipc` mehr Bits? Was würde sich ändern, wenn `auipc` ebenfalls nur 12 Bit nutzen könnte? Wie könnten Sie dieser Änderung entgegenwirken?
3. Malte hat ein Assembly-Programm geschrieben, um sich mit der Speicherstruktur von RISC-V vertraut zu machen:

```

1  .data
2      zahl1: .word 0x0ca7be5a
3      zahl2: .word 0xbaedfeed
4
5  .text
6      la t0 zahl1
7      addi t0 t0 2
8      pnmbokibvcy7sxz6rewq

```

Leider ist seine Katze über die Tastatur gelaufen und hat die Zeile 8 verunstaltet. Er weiß noch, dass er entweder ein Byte, Half word oder Word von der Adresse in t0 in t1 geladen hat und die Ausgabe in t1 am Ende 0xfeed0ca7 war.

- a) Welche Ausgaben erhalten Sie jeweils für das Laden eines Bytes, Halbwortes und Wortes? Wie ändert sich das Ergebnis im Allgemeinen für ein unsigniertes Laden?
- b) Wir ändern die Instruktion in Zeile 7 zu `addi t0 t0 3`. Wie ändert sich das Alignment im Vergleich zu vorher?
- c) Wie kann trotz ungültigen Alignment auf Speicher zugegriffen werden? Warum sollten Sie dies vermeiden?

Aufgabe 3 *Load-Store Architektur*

1. Die folgenden Instruktionen stammen aus einer Speicher-Speicher-Architektur und werden nicht in einer Load-Store Architektur unterstützt. Schreiben Sie äquivalente Programme in der RISC-V ISA. [A], [B], [C] sind Speicheradressen.
 - a) `MUL [A], [B], [C]` (Multiplizieren der Werte an den Speicheradressen [B] und [C]; Schreiben des Ergebnisses in [A])
 - b) `MOV [A], [B]` (Kopie zwischen zwei Speicheradressen)
 - c) `REDUCE_SUM [A], [B], [C]` (Addieren der Werte im Speicherbereich zwischen [B] und [C] und schreiben des Ergebnisses nach [A])
2. Welche Vor- und Nachteile hat eine Load-Store Architektur in diesem Kontext? Diskutieren Sie!
3. Welche architektonischen Änderungen wären an der RISC-V Architektur nötig, um `MOV [A], [B]` in einer Instruktion zu ermöglichen?

Aufgabe 4 *Pulsbreitenmodulation in Ripes*

In dieser Aufgabe wollen wir Pulsbreitenmodulation in Ripes umsetzen. Dazu richten wir zunächst die benötigten Ein- und Ausgabegeräte ein.

1. Öffnen Sie in Ripes den **I/O**-Tab in der linken Seitenleiste.
2. Fügen Sie im Bereich **Devices** durch Doppelklick eine **LED-Matrix** hinzu. Auf der rechten Seite können Sie nun folgende Einstellungen vornehmen:
 - Höhe: 1
 - Breite: 1
 - Pixelgröße: 100

Anschließend erscheinen links die Konstanten `LED_MATRIX_0_BASE` und `LED_MATRIX_0_SIZE` mit dem Wert `0x4`. Diese können später im Programmcode verwendet werden. Da wir eingestellt haben, dass die LED-Matrix nur ein Pixel besitzt, genügt es `0xffffffff` für weiß und `0x000000` für schwarz

an die Speicheradresse `LED_MATRIX_0_BASE` schreiben, um zwischen an und aus zu wechseln.

3. Fügen Sie nun durch Doppelklick im Bereich **Devices** ein **D-Pad** hinzu. Auf der rechten Seite sehen Sie, mit welchem Offset relativ zur Konstante `D_PAD_0_BASE` die Zustandswerte der Tasten gespeichert werden. Je nach Tastendruck werden dabei die Speicherwörter `0x0` bzw. `0x1` an den Offsets gesetzt. Diese können Sie also dynamisch im Programm aus dem Speicher auslesen.
4. Innerhalb Ihres Programmes können Sie diese Konstanten zum Beispiel mit `li t0 LED_MATRIX_0_SIZE` in ein Register laden.

Ihr Programm soll über folgende drei Konstanten steuerbar sein, welche im Datensegment definiert werden. Diese Werte sind allerdings nicht in Stein gemeißelt. Probieren Sie also gerne aus mit welcher Kombination der Effekt am Besten erkennbar ist. Erklären Sie, warum auf verschiedenen Geräten der Effekt bei gleichen Einstellungen unterschiedlich gut zu erkennen ist.

1. **pulseLength** := 1000: `pulseLength` gibt die Gesamtdauer einer Periode an. Falls Sie Probleme mit flackerndem Licht haben, empfiehlt es sich einen höheren Wert von z.B. 10000 zu wählen. Dadurch ist allerdings der Effekt der Pulsbreitenmodulation schlechter sichtbar.
2. **inputSpeed** := 20: Anzahl der Schleifendurchläufe, die gewartet wird, bevor Eingaben erneut ausgewertet werden.
3. **initialBrightness** := 40: Helligkeit zu Beginn der Programmausführung. Die Helligkeit liegt zwischen 0 und `pulseLength` und gibt an, ab welchem Zeitpunkt innerhalb einer Periode das Pixel von schwarz auf weiß wechselt.

Versuchen Sie an dieser Stelle, das Assembly-Programm selbst zu entwerfen. Der Effekt wird erst mit den gegebenen Konstanten sichtbar, wenn Sie das Programm im Modus `Fast Execution` (also durch Klicken auf den Doppelpfeil) ausführen. Falls Sie nicht wissen, wie Sie beginnen sollen, finden Sie im Anschluss Hinweise zur Strukturierung der Lösung. Diese müssen Sie allerdings nicht eins zu eins befolgen. Es kann ebenso helfen die Programmstruktur zunächst mit Pseudocode oder in einer anderen Programmiersprache zu planen.

Grundlegende Hinweise zur Programmstruktur

- Strukturieren Sie das Programm mithilfe von Sprungmarken, z. B.:
 - `start`
 - `tick`
 - `checkInput`
 - `increaseBrightnessTestBounds`
 - `increaseBrightness`
 - `decreaseBrightnessTestBounds`
 - `decreaseBrightness`

- drawPixel
- removePixel
- paintCanvas
- Folgende Belegung von Saved Registers kann hilfreich sein:
 - s_0 : Aktuell eingestellte Helligkeit
 - s_1 : Anzahl der verbleibenden Aufrufe von Tick für die aktuelle Periode
 - s_2 : Anzahl der verbleibenden Aufrufe von Tick bis innerhalb der aktuellen Periode die Ausgabe auf schwarz gesetzt wird
 - s_3 : Anzahl der verbleibenden Aufrufe von Tick bevor erneut eine Eingabe verarbeitet wird
 - s_4 : Farbe, die durch paintCanvas angezeigt werden soll

Die temporary registers t_0 bis t_6 können ebenfalls innerhalb des Programms verwendet werden. Allerdings sollten die darin gespeicherten Daten nur innerhalb des aktuellen Segments verwendet werden. Bedenken Sie, dass Ihr Programm von einer beliebigen Stelle zu einer Sprungmarke springen kann und Sie somit nach einer Sprungmarke nicht wissen, welche Werte in den temporären Registern gespeichert sind.

Ausführliche Hinweise zur Programmstruktur:

- **.data:** Hier sollten pulseLength, inputSpeed, initialBrightness definiert werden.
- **start:** Die initialen Werte für s_0 , s_1 , s_2 und s_3 werden gesetzt.
- **tick:**
 - Die Anzahl der verbleibenden Aufrufe von Tick für die aktuelle Periode wird reduziert. Falls der Wert 0 erreicht ist, wird zu drawPixel gesprungen.
 - Die Anzahl der verbleibenden Ticks, bis die Ausgabe auf schwarz gesetzt wird, wird reduziert. Falls der Wert 0 erreicht ist, wird zu removePixel gesprungen.
 - Die Anzahl der verbleibenden Ticks, bis die Eingabe verarbeitet wird, wird reduziert. Falls der Wert 0 erreicht ist, wird zu checkInput gesprungen.
 - Es wird zum Beginn von tick gesprungen.
- **checkInput:** Die Anzahl der verbleibenden Ticks wird zurückgesetzt. Es wird die Adresse D_PAD_0_BASE und die Werte an den korrekten offsets aus dem Speicher geladen. Diese werden mit 1 verglichen. Falls der Test positiv war, wird entweder increaseBrightnessTestBounds oder decreaseBrightnessTestBounds aufgerufen. Sonst wird zu tick zurückgesprungen.
- **increaseBrightnessTestBounds:** Falls die aktuelle Helligkeit kleiner ist als pulseLength minus 1 wird increaseBrightness aufgerufen. Sonst wird zurück zu tick gesprungen.

- **increaseBrightness:** Die aktuelle Helligkeit wird um 1 erhöht und es wird zu `tick` zurückgesprungen.
- **decreaseBrightnessTestBounds** und **decreaseBrightness** funktionieren analog zur Erhöhung der Helligkeit. Allerdings sollte mit 2 verglichen werden.
- **drawPixel:** Die Farbe weiß wird in das Register s_4 geladen. s_1 und s_2 werden zurückgesetzt. Anschließend wird zu `paintCanvas` gesprungen.
- **removePixel:** Die Farbe schwarz wird in das Register s_4 geladen. Anschließend wird zurück zu `paintCanvas` gesprungen.
- **paintCanvas:** Die Farbe aus Register s_4 wird an die Speicheradresse `LED_MATRIX_0_BASE` geschrieben. Anschließend wird zurück zu `tick` gesprungen.